Evidence-Based Linux Game-Performance Tuning with `stutter`

> **Draft status:** this is the long-form report draft/source, not the first supervisor handoff. For supervision outreach, send `FYP_SUPERVISOR_PITCH.md` and `PRE_FYP_SCOPE_NOTE.md` first; this report is retained as a deeper technical draft until the assessed scope is agreed.

1. Abstract

Linux game-performance tuning is often based on anecdotal tweaks and short play sessions. This project presents `stutter`, a prototype Linux profiling and tuning tool that combines eBPF scheduler evidence, process-tree tracking, MangoHud frame timing, structured artifacts, guarded tuning hypotheses, and repeated A/B validation. The tool records a workload, analyses scheduler and frame-pacing evidence, proposes scoped tuning experiments, explains which tasks a profile would affect, and declines to recommend changes that are not supported by repeated measurement.

The prototype is evaluated through a case study on Kingdom Come: Deliverance 1 under GE-Proton10-34 on Wayland/Gamescope. The case study tested a CPU-affinity hypothesis that moved game/Wine tasks to CPUs `1-5,7-11` and Gamescope/runtime tasks to CPUs `0,6`. Across five paired A/B iterations, the online baseline had a lower primary diagnostic score than the tuned profile in every iteration. The archived run was paired but not counterbalanced: the online baseline was measured before the tuned profile in every iteration. The profile was therefore not validated on the current evidence, but the result should be treated as caveated low-confidence evidence rather than a fully counterbalanced effect estimate.

The result demonstrates the value of evidence-based tuning: the tool did not produce a general Linux performance tweak, but it successfully prevented a plausible false-positive recommendation in a noisy real-world workload.

2. Introduction

2.1 Problem Statement

Linux game performance is shaped by many interacting layers: the kernel scheduler, CPU topology, compositor, GPU driver, Wine/Proton runtime, Gamescope, MangoHud, Steam runtime processes, and the game engine itself. A user may see acceptable average FPS while still experiencing uneven frame pacing, p99 frametime spikes, audio or input disruption, or presentation stalls. Average FPS is therefore an incomplete measure of smoothness.

Linux gaming communities often respond to these issues with tuning advice: change CPU affinity, change scheduler settings, add launch flags, change compositor behavior, alter memory allocator choices, or use driver-specific options. Some of this advice may be useful in a particular setup. The problem is that it is rarely validated under controlled conditions, and noisy open-world games can make a weak change appear helpful after only one run.

This project addresses that gap by treating tuning advice as a testable hypothesis. A tool should collect evidence, make a scoped proposal, explain what the proposal would change, test it repeatedly, and refuse to recommend it when the evidence does not support the change.

2.2 Project Aim

The aim is to build and evaluate a prototype that turns Linux game-tuning advice into a measurable workflow. The prototype, `stutter`, is not intended to be a general Linux gaming tweak. It is an evidence tool: record, diagnose, propose, explain, test, and decline unsupported recommendations.

2.3 Research Question

The research question is:

> Can a Linux profiling prototype collect enough evidence from a real Proton game workload to generate, explain, and validate or decline to recommend a scoped tuning hypothesis?

This wording matters. The goal is not only to find improvements. A responsible tool must also avoid recommending a plausible tweak when repeated measurements do not support it.

2.4 Contributions

The project makes the following contributions:

- A scheduler-aware recording pipeline using Rust-based eBPF instrumentation.
- Frame-timing correlation using MangoHud CSV data.
- Process-tree and task classification for Proton/Wine/Gamescope workloads.
- An advisor and tuning workflow for guarded profile experiments.
- A profile explainability path for rule-level task matching.
- A real KCD1 case study showing evidence-based non-validation of a plausible CPU-affinity tweak.
- A validation corpus and repository check flow that support regression testing beyond the single case study.

2.5 Report Structure

The report first introduces the background concepts needed to understand game stutter, runnable latency, eBPF tracing, Proton process trees, CPU affinity, and A/B testing. It then describes the requirements, architecture, implementation, and experimental methodology of `stutter`. The central evaluation chapter presents the KCD1 case study. The final chapters separate results, discussion, limitations, future work, references, and reproducibility appendices.

3. Background and Related Concepts

3.1 Frame Pacing and Stutter

Frames are perceived over time, so smoothness depends on consistency as well as average rate. Average FPS compresses a whole run into one number. Frametime instead measures how long each frame takes. Stutter often appears in p95, p99, maximum frametime, or outlier counts rather than in the mean.

For example:

```
At 100 FPS, the expected frame interval is about 10ms.
A 50ms frame is a multi-frame stall even if the average FPS remains high.
```

This is why the project focuses on frame-pacing tails, MangoHud frame logs, and outliers [11]. A tuning change that leaves mean FPS similar but reduces p99 frametime could be useful. Conversely, a change that moves one frame metric in a favorable direction while increasing scheduler tail latency may not be safe to recommend.

3.2 Linux Scheduling and Runnable Latency

Linux tasks become runnable when they have work to do and are eligible to run on a CPU. Runnable latency is the delay between a task becoming runnable and actually receiving CPU time. Linux scheduler documentation provides the background for this model [4]-[6]:

```
sched_wakeup timestamp -> sched_switch timestamp = runnable latency
```

High runnable latency can matter for game threads, render threads, DXVK submit threads, streaming workers, audio threads, Wine helper processes, or compositor tasks. It should not be read as a claim that every visible stutter is scheduler-related. It does provide scheduler-visible evidence that can be combined with frame timing and process identity.

Games under Proton produce many Linux-visible tasks: the game process, Wine services, DXVK-related threads, Gamescope, Steam runtime processes, and helper tasks. That makes

process-tree tracking and classification necessary. A single PID is usually not enough context for a useful tuning decision.

3.3 eBPF as a Measurement Tool

eBPF allows a user-space program to attach safe, verified programs to selected kernel events. For `stutter`, this makes scheduler-visible timing practical: events can be observed near the source rather than inferred through coarse user-space polling. The result is lower-latency evidence about wakeups, context switches, and related runtime behavior. The implementation model follows the Linux BPF, map, and ring-buffer documentation [1]-[3].

The same mechanism has limitations. eBPF programs use maps and buffers with finite capacity. Optional probes may not be available on every kernel or setup. Counters such as replacement counts or ring-buffer reserve failures need careful interpretation. The KCD1 drop-counter pilot later shows why this matters: non-zero wakeup replacement counters were not the same as ring-buffer event loss.

3.4 Proton, Wine, DXVK, and Gamescope

The KCD1 workload is mediated by a layered Linux gaming stack [7]-[13]:

```
Windows game
  -> Proton/Wine
  -> DXVK/Vulkan/RADV render path
  -> Gamescope presentation/compositor layer
  -> Wayland/Sway session
  -> GPU/display
```

Each layer can create tasks that appear in Linux scheduler evidence. Wine may spawn helper processes and `wineserver`. DXVK may create submit, queue, or shader-related threads. Gamescope contributes compositor/runtime work. Steam runtime tasks and launchers can also appear around the game.

Thread naming is also complicated. A thread's `task.comm` can differ from its parent process `process_comm`. In the KCD1 profile, a broad `match_comm = ["Main"]` rule could match worker threads whose own names were `RenderThread`, `ClothingRaycast`, `Streaming Async`, or `dxvk-submit`, because those threads belonged to a process whose `process_comm` was `Main`. This is why profile explainability became important.

3.5 CPU Affinity and Tuning Risks

CPU affinity restricts where tasks are allowed to run. It can be useful when a workload benefits from separating classes of work, reducing interference, or keeping runtime tasks away from game threads. It can also reduce scheduler flexibility and effective CPU capacity.

That trade-off is central to the KCD1 case study. On a 6-core/12-thread CPU, reserving one SMT pair (`0,6`) for Gamescope/runtime work removes a meaningful portion of the logical CPU set from the game side. A plausible profile can therefore become unsupported, or unsuitable in a specific setup, if it compresses a busy game process onto too few CPUs. This is exactly why the profile needed repeated validation instead of being recommended from diagnosis alone.

3.6 Why Repeated A/B Testing Matters

A/B testing compares a baseline and a candidate under similar conditions. Interleaving baseline and tuned runs helps reduce time drift, and repeated measurements help account for workload variance. This is especially important for open-world games where traversal, asset streaming, shader state, and background runtime work can vary. The report uses confidence intervals and sample-size estimates as conservative measurement tools rather than as proof of causality [14]-[16].

The trusted tuning loop in `stutter` is:

```
diagnosis candidate -> structured fix hypothesis -> validation experiment -> A/B evidence -> f
```

Confidence intervals, sample-size estimates, data-quality checks, and verdicts such as `NeedsRetest` prevent overclaiming. The correct result is not necessarily "validated". Sometimes the correct result is "not enough evidence" or "do not recommend this profile".

4. Requirements and Design Goals

4.1 Functional Requirements

The prototype is expected to:

- Record scheduler, frame, process-tree, GPU, and data-quality evidence.
- Associate evidence with a selected workload or process tree.
- Ingest MangoHud frame logs.
- Summarise performance and stutter evidence.
- Classify relevant game, Wine, runtime, compositor, and helper tasks.
- Generate tuning hypotheses from recorded evidence.
- Represent candidate changes as profile or fix-plan artifacts.
- Run profile-based A/B tuning experiments.
- Produce reports and machine-readable artifacts.
- Explain profile matching before or after application.

4.2 Non-Functional Requirements

The prototype should also satisfy non-functional goals:

- Low enough overhead for real game workloads.
- Reproducible artifacts that can be inspected after the run.
- Stable JSON, NDJSON, Markdown, and HTML outputs where applicable.
- Clear diagnostics and cautious language.
- A CLI usable by technical Linux users.
- Robustness in noisy workloads.
- Separation between observation, suggestion, and mutation.

4.3 Safety Requirements

The safety model is a core design requirement, not an afterthought:

- Apply supported low/medium-risk actions only through policy-controlled paths, with preflight checks and rollback requirements where applicable.
- Provide dry-run modes for profile inspection.
- Gate medium-risk actions through explicit policy.
- Require user intent or force flags where persistent effects are possible.
- Avoid hidden persistent changes.
- Preserve rollback state when a supported action is applied.
- Avoid recommending unsupported tweaks as validated fixes.
- Treat data-quality failure as a reason to block or revert an experiment.

This language is deliberately cautious. Some actions are reversible, some are suggest-only, and some are blocked by policy. The report should not imply that every possible tuning action is safe to apply.

4.4 Evaluation Requirements

The project must be evaluated as both software and methodology:

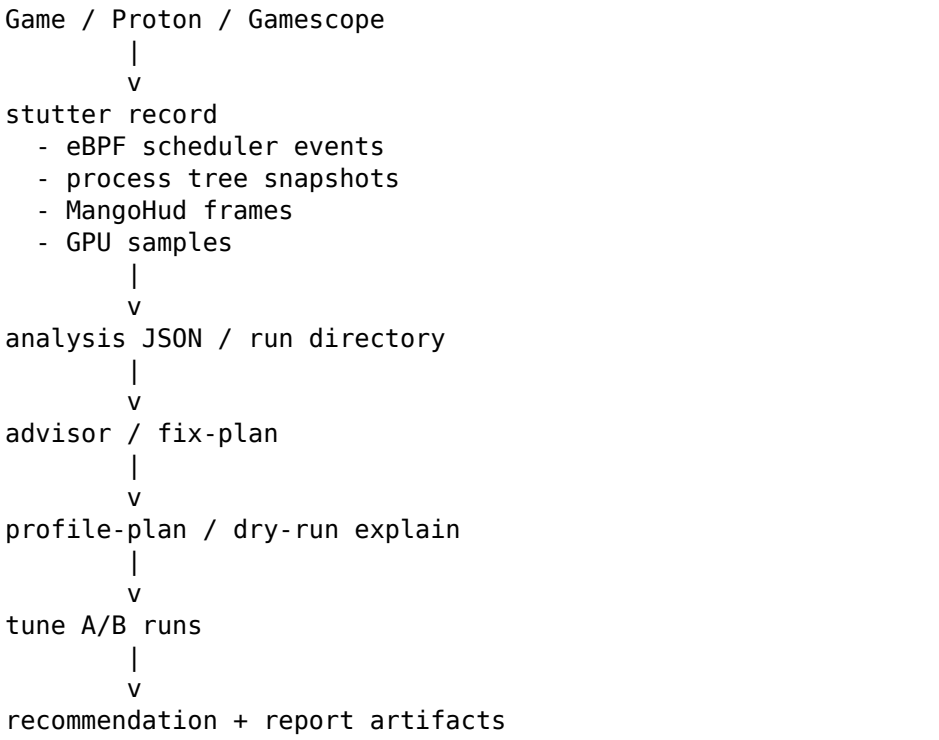
- Validate code with formatting, tests, linting, and fixture checks.
- Validate artifacts with structured parsing and review.
- Check that large or ignored generated files are not accidentally staged.
- Use a real workload case study rather than only synthetic fixtures.
- Preserve raw and derived evidence.
- Show that a plausible recommendation can be declined when data does not support it.

5. System Architecture

5.1 Architecture Overview

`stutter` is organized as a pipeline from evidence collection to cautious recommendation:

Figure 1. Evidence-to-recommendation pipeline:



The important architectural boundary is that recording and analysis do not mutate the machine. Planning produces candidates. Action execution is guarded by policy, preflight checks, and rollback requirements.

5.2 Recorder Pipeline

`stutter record` is the evidence collector. In the KCD1 case study it targets a live process tree with `--tree-pid`, records for a configured duration, and writes a run directory rather than one monolithic file.

Typical run artifacts include:

- `metadata.json`
- `session.json`
- `tree_events.json`
- `spike_events.json`
- `frame_correlation.json`
- `gpu_samples.json`
- `cpu_freq_samples.json`
- `runtime_slices.json`
- `foreground_events.json`
- `focus_events.json`
- `interval.json`

For example, KCD1 baseline runs such as `${KCD1_RAW}/runs/baseline-01` preserve both raw and derived evidence. This makes later claims auditable.

5.3 Analysis and Diagnostic Score Pipeline

The analysis pipeline converts event streams into summaries. It identifies scheduler spikes, frame-pacing outliers, task attribution, data-quality warnings, and comparison metrics. For the KCD1 case study, the key comparison metric is `diagnostic_raw_score_total`.

The simplified frame-aware score shape is:

Figure 2. Simplified Diagnostic Score shape:

```
scheduler component:
  over_5ms * 100
+ over_2ms * 20
+ over_1ms

frame component:
  frame_over_50ms * 100
+ frame_over_33ms * 20
+ frame_over_16ms
```

This score is useful only for comparable runs under the same workload, route, measurement settings, and analysis path. It is not FPS and not a general-purpose performance score.

5.4 Advisor and Fix-Plan Pipeline

The advisor turns diagnosis into candidate actions. In the KCD1 case study, the advisor produced a CPU-affinity hypothesis: move game/Wine work away from the Gamescope/runtime CPU pair. The output is a scoped experiment proposal, not a truth claim.

Fix plans are structured hypothesis artifacts. They can include evidence, expected effect, safety class, rollback expectations, affected scope, and validation requirements. A fix plan must still be tested through repeated comparison before becoming advice.

5.5 Tune/Recommend Pipeline

`stutter tune` runs repeated profile comparisons. A tune profile set includes `baseline-online` so the tuned profile can be compared against an explicit online-mask control. Warmup and measurement periods are separated so startup or route stabilization does not contaminate the measured window.

`stutter recommend` interprets the result. A useful recommendation pipeline can return `NeedsRetest`, underpowered, inconclusive, or invalid-experiment results. That is not a failure of the tool. It is how the tool prevents weak evidence from becoming advice.

5.6 Profile Explainability Pipeline

Profile explainability was added because the KCD1 case study exposed a reporting gap. The original dry-run could show that tasks would move, but not enough about why each task matched. `profile-plan` and `apply-profile --dry-run --explain` fill that gap.

The explainability path reports:

- rule-level matched task counts;
- proposed affinity masks;
- `task.comm` and `process_comm`;
- task class;
- match source or basis;
- first-match-wins behavior;
- broad `process_comm` captures;
- highlighted comms for important threads.

This is an FYP contribution because explainability is also a safety feature. Users need to know what a profile will move before trusting a tuning experiment.

5.7 Artifact Model

The project keeps raw and derived artifacts. JSON files represent structured state and summaries. NDJSON-style streams represent event sequences where appropriate. Optional streams may be absent or empty depending on flags, kernel support, and workload conditions.

The artifact model makes the case study reproducible. Instead of relying on a memory of a live run, the report can point to baseline directories, tune summaries, recommendation JSON, profile-plan output, setup notes, and artifact indexes.

6. Implementation

6.1 Rust Workspace Structure

The repository is a Rust workspace with these members:

Table 1. Rust workspace crates:

Crate	Role
`stutter`	Main CLI, recording, analysis orchestration, commands
`stutter-common`	Shared common structures
`stutter-core`	Core typed primitives
`stutter-config`	Configuration model and effective config resolution
`stutter-ebpf`	eBPF-side code and build integration
`stutter-report`	Report model, loading, analysis, diffing, rendering
`xtask`	Repository validation and maintenance commands

This layout keeps the tracing, artifact model, report rendering, configuration, and repository checks from collapsing into one unstructured binary.

Relevant implementation anchors include:

- recording session setup, writers, and finalization in `stutter/src/recorder/session/prepare.rs`, `stutter/src/recorder/session/writers.rs`, and `stutter/src/recorder/session/finalize.rs`;
- runnable-latency capture and wakeup replacement accounting in `stutter-ebpf/src/scheduler.rs`, `stutter-ebpf/src/wakeup_data.rs`, `stutter-ebpf/src/maps.rs`, and `stutter-ebpf/src/drop_counter`
- MangoHud parsing and timestamp alignment in `stutter/src/mangohud/parser.rs` and `stutter/src/mangohud/alignment.rs`;
- report frame analysis in `stutter/src/report/analysis/frame.rs`;
- profile matching and explainability in `stutter/src/profiles/matching.rs`, `stutter/src/profiles/explain.rs`, `stutter/src/tune/profile_plan.rs`, and `stutter/src/watch/profile`
- tune execution, ranking, statistics, and recommendations in `stutter/src/tune/run.rs`, `stutter/src/tune/ranking.rs`, `stutter/src/tune/statistics.rs`, and `stutter/src/tune/recommenda`
- fix-validation rendering in `stutter/src/recommend/fix_validation.rs` and `stutter/src/recommend/render.rs`;
- policy and rollback paths in `stutter/src/actions/runner/policy.rs`, `stutter/src/actions/runner/r` `stutter/src/watch/policy.rs`, and `stutter/src/profile_restore/`;
- validation fixtures and golden reports in `xtask/src/fixtures.rs`, `xtask/src/fixture_coverage.rs`, and `stutter-report/tests/golden.rs`.

6.2 Process-Tree Classification

The process-tree model is needed because Proton games create multiple relevant Linux tasks. The tool needs to distinguish game threads, helpers, WineServer, Gamescope, Steam runtime tasks, compositor work, and unknown tasks.

KCD1 shows why process names alone are not enough. A process can be named `Main`, while important worker threads have separate `task.comm` values such as `RenderThread`, `ClothingRaycast`, `Streaming Async`, or `dxvk-submit`. Classification and profile matching therefore need both thread-level and process-level context.

6.3 eBPF Event Capture

The eBPF path captures scheduler-visible timing and emits event streams for the recorder. Conceptually, it observes wakeup and switch timing so `stutter` can measure runnable latency. It also tracks drop and replacement counters so that measurement quality can be interpreted.

This comes with practical constraints. The process needs privileges to load eBPF programs on most systems. Kernel support can vary. Map capacity and ring-buffer capacity must be sized carefully. The KCD1 drop-counter pilot is included because it distinguishes replacement churn from ring-buffer reserve failure.

6.4 Frame Correlation

Frame correlation uses MangoHud CSV data. The recorder ingests frame timing and aligns it with the run timeline. In the formal KCD1 baseline set, the frame timestamp alignment was `monotonic_observed`, which made the frame data usable for repeated comparison.

Frame metrics include frame counts, median frametime, p95, p99, maximum frametime, and outlier counts. These metrics are interpreted alongside scheduler evidence rather than replacing it.

6.5 Profile Matching

Profiles are TOML-defined collections of ordered rules. First matching rule wins. Rules can match task classes, task `comm`, or process `process_comm`, depending on the match type. The KCD1 profile used:

- a baseline profile that kept tasks on the online CPU set;
- a tuned profile that placed `Main` and game/Wine classes on `1-5,7-11`;
- a Gamescope/runtime rule that placed those classes on `0,6`.

The first-match-wins rule is important because a broad early match can prevent later, more specific rules from applying. That is why the report treats profile explainability as necessary rather than optional.

> **Profile matching caveat:** Profile rules are first-match-wins. A broad rule such as `match_comm = ["Main"]` can match many worker threads through `process_comm = "Main"` before later, more specific rules are considered. This is why `profile-plan` or `apply-profile --dry-run --explain` should be run before applying or tuning a profile.

6.6 Explainability Model

The explainability model reports what a profile would do before it is applied. Conceptually it contains:

- report-level summary counts;
- rule summaries;
- per-task matches;
- original and proposed masks;
- matched task classes;
- match source or basis;
- broad `process_comm` capture detection;
- highlighted comms for important threads.

For KCD1, this showed that key game/DXVK/Wine worker threads were matched by the broad `process_comm = "Main"` behavior. That made the A/B result easier to interpret because the tuned profile did not fail merely by missing the target threads.

6.7 Safety and Policy Checks

The implementation separates dry-run explanation, one-shot apply, watch mode, and persistent effects. For example, `apply-profile --explain` requires `--dry-run`, dry-run is not combined

with watch mode, medium-risk policies are gated, and persistent effects require explicit user intent.

The wider safety model requires action descriptors, preflight checks, rollback requirements where applicable, policy checks, audit events, and verification before an applied experiment is kept. This is why the report describes `stutter` as a guarded experiment tool rather than an auto-tweaker.

6.8 Testing and Validation

Testing covers both code behavior and artifact behavior. The repository uses:

- formatting checks with `cargo fmt`;
- unit and integration tests with `cargo test`;
- linting with `cargo clippy`;
- report golden tests;
- architecture tests;
- validation-corpus fixtures;
- `xtask fixture-check`.

The validation corpus is especially relevant to the FYP because it covers more than the KCD1 case study: real and synthetic runs, multiple vendors, multiple compositors, known false positives, known false negatives, and data-quality cases.

7. Experimental Methodology

7.1 Measurement Principles

The experimental method follows these rules:

- Keep the workload stable.
- Change one main variable at a time.
- Use repeated runs.
- Preserve raw and derived evidence.
- Separate warmup and measurement windows.
- Avoid causal claims when many variables changed.
- Treat uncertainty as a result, not as an inconvenience.

7.2 Workload Selection

KCD1 is a useful workload because it is a real open-world game running through Proton/Wine. It has a complex process tree, frame-pacing variation, and enough runtime complexity to expose scheduler and profile-matching issues. It is also noisy, which makes it a good test of whether the tool avoids overclaiming.

7.3 Baseline Collection

The formal baseline set used five runs of a repeatable Rattay route from the same save. Each run lasted about 180 seconds. The setup used Gamescope and MangoHud frame logging, explicit process-tree targeting, and a stripped-down launch configuration so the CPU-affinity profile would be the main variable later. The exact command shapes are listed in Appendix A; live process IDs were re-detected before each run.

Baseline validity checks included stop reason, duration, frame count, timestamp alignment, data quality, and artifact completeness.

7.4 Hypothesis Formation

The advisor suggested a CPU-affinity profile because the baseline showed scheduler-visible latency and frame-pacing outliers. The hypothesis was that placing Gamescope/runtime work on CPU pair `0,6` and game/Wine work on `1-5,7-11` might reduce interference.

This was a plausible hypothesis, not a conclusion. It was reversible and therefore suitable for a profile experiment.

7.5 A/B Tuning Design

The tune run compared `baseline-online` against the tuned profile. Each profile had five measured iterations. The tune command used a 90-second warmup and a 270-second epoch so that each epoch provided 180 seconds of measurement after warmup. The generated summary recorded restore behavior after each profile. Review of the archived `candidate_order` showed that `baseline-online` was measured before the tuned profile in every iteration. The KCD1 run was therefore paired but not counterbalanced, and future methodology must enforce alternating AB/BA order for two-profile comparisons.

This design prevents a profile from being recommended solely because it was a good story. It must perform better under repeated measurement.

7.6 Data-Quality Checks

The quality checks include:

- stop reason;
- duration;
- frame count;
- timestamp alignment;
- data-quality level;
- drop and replacement counters;
- missing optional correlations;
- run comparability.

The formal KCD1 runs were usable, but not lab-perfect. They were treated as valid with limitations rather than as clean synthetic benchmarks.

7.7 Interpretation Rules

The interpretation rule is:

```
A profile is not recommended unless evidence supports it across repeated
comparisons. NeedsRetest is not failure; it is a correct uncertainty result.
```

The report therefore uses careful wording: not validated, not recommended on current evidence, plausible hypothesis, likely explanation, and observed in this workload. It avoids claiming general behavior beyond the measured setup.

8. KCD1 Case Study

> **Raw KCD1 artifact note:** `${KCD1_RAW}` denotes the `reports/kcd1-case-study/` directory inside the extracted `raw-kcd1-case-study-fyp-report-final-v4.tar.zst` release asset. Large raw run, tune, MangoHud, setup, profile, and exploratory artifacts are not tracked in the cleaned Git repository.

8.1 Setup

The main evaluation used Kingdom Come: Deliverance 1 under Steam with GE-Proton10-34, Sway/Wayland, Gamescope, and MangoHud frame logging.

Table 2. Experimental setup:

Item	Value
Game	Kingdom Come: Deliverance 1
Platform	Steam + GE-Proton10-34
Session	Gentoo Linux, Sway/Wayland, Gamescope
CPU	Intel i5-10600K, 6 cores / 12 threads

Item	Value
GPU	AMD Radeon RX 9070 XT
Route	Repeatable Rattay route from the same save
Duration	180 seconds per measured run
Baselines	5 formal baseline runs
A/B test	5 `baseline-online` + 5 tuned profile runs
Main variable	CPU-affinity profile

The measurement launch kept Gamescope, MangoHud logging, and the archived KCD1 config. It excluded the author's larger personal optimized configuration: no RADV experimental flags, no FSR/FSR4, no gamemode, no mimalloc, and no forced Wine CPU topology. Appendix A lists command shapes, Appendix B gives the tested profile TOML, Appendix C maps the artifacts used as evidence, and Appendix D records the reproducibility checklist.

8.2 Baseline Results

Five formal baselines passed the basic validity checks: each ran for about 180 seconds, stopped because the maximum duration was reached, ingested MangoHud frame data, used monotonic timestamp alignment, and reported `Medium` data quality.

Table 3. Baseline frame timing:

This table is derived from the formal baseline analysis artifacts in `${KCD1_RAW}/runs/`. Frametime units are milliseconds, and the outlier count is `frame_pacing.outlier_count`: frames at or above 33.3ms, or at least 2x the run median frametime.

Run	Frames	Median frametime	P99	Max	Outliers
baseline-01	8,833	17.725ms	51.008ms	562.266ms	1,347
baseline-02	7,744	21.276ms	49.712ms	272.166ms	1,354
baseline-03	9,261	16.309ms	49.085ms	563.179ms	1,303
baseline-04	7,421	22.811ms	46.778ms	84.384ms	1,229
baseline-05	7,607	22.884ms	44.788ms	87.387ms	1,098

The baseline set was valid but noisy. Median frametime varied from about 16.3ms to 22.9ms, while p99 remained in the mid-40s to low-50s milliseconds. That supported repeated A/B measurement rather than a one-run tuning claim.

8.3 Advisor Hypothesis

The advisor generated a plausible CPU-placement hypothesis: reserve the core-0 SMT pair for Gamescope/runtime work and place KCD1/Wine/game threads on the remaining CPUs. The hypothesis was motivated by scheduler-visible tail latency and frame-pacing outliers, but diagnosis alone did not validate it.

The profile rules were:

Table 4. CPU-affinity profile rules:

Rule	Match	Affinity	Intended effect
0	`match_comm = ["Main"]`	`1-5,7-11`	Move KCD process threads
1	`Game`, `GameHelper`, `WineServer`	`1-5,7-11`	Move Wine/game helpers
2	`GameScope`, `Compositor`, `Launcher`, `SteamRuntime`	`0,6`	Reserve core-0 SMT pair for presentation/runtime

8.4 Profile Explainability

The profile-plan follow-up showed what the tuned profile would do before application. In this case study, the profile-plan explainability pass was added as a follow-up after the A/B run exposed the need to audit which rules matched which KCD1 threads. Future studies should run it before tuning:

Table 5. Profile-plan task-count summary:

Item	Count
Snapshot tasks	181
Matched tasks	114
Pending affinity changes	114
Rule 0 matched tasks	88
Rule 1 matched tasks	25
Rule 2 matched tasks	1

The explainability artifact showed that important KCD/DXVK/Wine-side worker threads were matched through `process_comm = "Main"`, including render, streaming, DXVK, audio, shader, physics, and job-system threads. This means the tuned profile did not fail simply because it missed the relevant tasks.

8.5 A/B Results

The paired A/B tune run, `kcd1-affinity-02`, tested both profiles with five valid measured iterations each. The archived candidate order was fixed with `baseline-online` before the tuned profile in every iteration, so the run was not counterbalanced and may include an order effect. Scores are derived from `${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`; lower diagnostic score is better.

Table 6. A/B per-iteration diagnostic score:

Iteration	Baseline-online score	Tuned profile score	Delta
1	19,579	32,052	+63.7%
2	21,533	34,566	+60.5%
3	16,643	44,845	+169.5%
4	26,408	38,806	+46.9%
5	32,994	98,461	+198.4%

Iteration 5 was the most extreme tuned-profile outlier. It raised the tuned profile's mean diagnostic score substantially, so the median score is the more stable condition-level summary. The conclusion does not depend only on this outlier: `baseline-online` still had a lower diagnostic score in all five paired iterations.

Table 7. Profile-level summary:

This table summarizes the same `tuning_summary.json` artifact at profile level.

Profile	Valid runs	Median diagnostic score	Mean diagnostic score	Median frame P99	Mean scheduler >5ms
`baseline-online`	5	21,533	23,431.4	38.337ms	0.2
`kcd1-game-on-1-5-7-11-gamescope-on-0-6`	5	38,806	49,746.0	37.798ms	0.6

The `over-5ms` value is a scheduler-latency threshold count from the diagnostic score, not a frame-time threshold.

The median comparison is visible in a compact bar:

Figure 3. Median diagnostic-score comparison:

```
baseline-online median: 21,533  [=========           ]
tuned-profile median:   38,806  [================    ]  +80.2% worse
```

The tuned profile was not validated and should not be recommended on the current evidence. Because the pair order was fixed, this is a caveated low-confidence non-validation result rather than a fully counterbalanced A/B estimate.

The profile-vs-profile tables above are derived from `tuning_summary.json` candidate statistics. The regenerated `tuning_recommendation.json` selects `baseline-online` as best and compares it against the best valid non-baseline candidate, `kcd1-game-on-1-5-7-11-gamescope-on-0-6`. The tuned-profile conclusion is unchanged: the tuned profile still does not validate on the current evidence.

The generated recommendation also estimated that some metrics may require more runs per side to detect a 10% movement at the observed noise level:

Because the generated recommendation selected `baseline-online` as the lower-scoring profile, these run-count estimates should be read as noise/sensitivity estimates from the recommendation artifact rather than as a claim that the tuned profile would validate with that exact number of additional runs.

Table 8. Sample-size estimates:

The values are estimated run counts per profile side from the tuning recommendation artifact.

Metric	Estimated runs per side
`diagnostic_raw_score_total`	30
`frame_p99_ms`	30
`frame_over_16ms`	30
`frame_over_33ms`	30
`frame_over_50ms`	30
`max_latency_ns`	19

8.6 Measurement-Quality Pilot

A separate drop-counter pilot investigated the recurring wakeup replacement counter.

Table 9. Drop-counter pilot summary:

The rates are summarized from `${KCD1_RAW}/drop-counter-pilot/mapfactor-4-comparison.txt`.

Condition	Wakeup replacements/s	Ringbuf reserve failures
baselines	about 1436-1557/s	0
mapfactor-4 pilot	about 1568/s	0

The pilot showed that wakeup replacement counters were not ring-buffer reserve failures and were not reduced by `--ebpf-wakeup-map-factor 4`. The likely interpretation is wakeup timestamp churn from rapid repeated wakeups for the same target tasks.

8.7 Exploratory Personal-Stack Comparison

An exploratory add-on compared the stripped-down measurement stack against the author's normal gaming configuration bundle, including `scx_lavd` and many launch flags. This changed many variables at once, so it is not a causal test of one flag or scheduler.

Table 10. Exploratory personal-stack comparison:

This table is derived from `${KCD1_RAW}/realworld-stack/realworld-stack-summary.csv`; it is a bundle comparison and not a single-variable causal test.

Condition	Runs	Median frame-time	P95	P99	Max	Median outlier %	Scheduler
clean	3	19.3419ms	26.2893ms	29.8236ms	65.7529ms	0.428%	default
personal-stack	3	20.1032ms	28.3838ms	32.4916ms	91.134ms	0.826%	`scx_lavd`

The `Max` column is the median of each condition's per-run maximum frametime, not the single worst frame observed across the condition. The worst individual personal-stack run was `personal-stack-02`, with a 181.596ms maximum frametime and a 4.814% outlier rate.

The personal stack did not show a clear advantage in this small sample. The value of the add-on is realism: `stutter` can capture and compare a complex player-used configuration bundle without overclaiming causality. As documented in the real-world stack artifact notes, two raw MangoHud CSV sources were not preserved, but their ingested frame data remains available in the archived analysis artifacts.

9. Results and Analysis

9.1 Main Result

The primary diagnostic score was lower for `baseline-online` in all five paired A/B iterations. This is the main evaluation result.

The result does not say that CPU affinity is inherently bad, or that the exact profile is unsuitable in every setup. It says that this profile did not earn a recommendation under this route, machine, Proton version, and measurement method.

9.2 Why the Tuned Profile Was Not Recommended

The tuned profile was not recommended because:

- it matched relevant KCD1 worker threads;
- it still had worse primary diagnostic scores;
- it likely reduced useful CPU capacity from 12 logical CPUs to 10 for the game side;
- the workload was noisy;
- the recommendation output correctly reported uncertainty through `NeedsRetest`.

The most plausible explanation is that reserving `0,6` for Gamescope removed too much useful scheduling capacity from KCD1/DXVK/Wine. That remains an interpretation, not a demonstrated causal mechanism.

9.3 What the Negative Result Demonstrates

The negative tuning result is not a failed project result. It validates the purpose of an evidence-based advisor: avoiding unsupported tuning advice. A tool that promotes every plausible hypothesis is not safe or useful in noisy game workloads.

9.4 What the Tool Learned

The case study revealed several engineering lessons:

- Profile explainability was needed to audit broad `process_comm` matches.
- Wakeup replacement counters needed careful interpretation and were not simply ring-buffer reserve failures.
- Recommendation output needed to distinguish candidate statistics from formal comparison fields when `baseline-online` was selected as the lower-scoring profile.

- Artifact context matters: setup notes, route labels, tune summaries, and profile plans are part of the evidence.

9.5 Threats to Validity

The main threats to validity are:

- one machine;
- one game;
- one route;
- one Proton version;
- missing IRQ/KMS/DRM optional correlations in the formal artifacts;
- explicit tree targeting rather than foreground auto-targeting;
- sample size too small for precise small-effect estimates;
- personal-stack comparison changing many variables at once.

These threats limit generalisation. They do not remove the central workflow result.

9.6 Evaluation Summary

The evaluation combines software validation and workload evidence. The codebase passed formatting, tests, linting, and fixture checks. The formal KCD1 baseline set produced valid run artifacts with MangoHud frame ingestion and monotonic timestamp alignment. The advisor produced a plausible CPU-affinity profile, and profile-plan output confirmed that the profile matched relevant game, Wine, and DXVK-side tasks. The repeated A/B tune then showed `baseline-online` with a lower primary diagnostic score in every paired iteration. However, the candidate order was fixed, so the result is treated as low-confidence non-validation evidence and the recommendation remains `NeedsRetest` rather than promoting the tuned profile. Together, these results support the report's central claim: `stutter` can collect evidence, form a guarded hypothesis, test it, and decline unsupported tuning advice while surfacing methodological limitations.

10. Discussion

10.1 Evidence-Based Tuning vs Tweak Guides

Online tweak advice is often anecdotal. It can be useful as a source of hypotheses, but it is not enough for a recommendation. `stutter` turns a tweak story into a measurement loop: record evidence, propose a scoped experiment, compare repeated runs, and report uncertainty.

10.2 Explainability as a Safety Feature

Explainability is a safety feature because a user needs to know what a profile will move. Broad process matches can capture more tasks than expected. First-match-wins rule order can make later rules irrelevant. Without explainability, a profile may appear simple while moving a large set of worker threads.

The KCD1 profile-plan artifact made the result more auditable. It showed that important worker threads were matched, which prevented a misleading explanation that the profile failed only because it missed the right tasks.

10.3 Why `NeedsRetest` Matters

A responsible tuning tool must be able to say "not enough evidence." `NeedsRetest` is useful because it prevents a ranking from becoming an overconfident recommendation when sample counts are low, noise is high, or confidence intervals cross zero.

10.4 Practical Value for Linux Gamers

The practical value of `stutter` is not that it gives every user a magic profile. It helps technical users:

- diagnose frame and scheduler evidence;
- avoid wasting time on unsupported tweaks;
- archive reproducible artifacts;
- compare configurations more honestly;
- understand which tasks a profile would affect.

10.5 Engineering Lessons

Real workloads expose gaps that synthetic tests can miss. KCD1 exposed the need for profile explainability and careful measurement-quality language. The project also shows that artifact hygiene, CLI safety, test coverage, and documentation are part of the tool, not afterthoughts.

11. Limitations and Future Work

11.1 Limitations

Table 11. Limitations:

Limitation	Impact
Prototype status	The tool still expects technical users and careful setup
One primary case study	Results should not be generalized to all games
One machine and route	Hardware and workload differences may change outcomes
One Proton version	Runtime updates could change process behavior
Missing IRQ/KMS/DRM correlation	Some display or interrupt causes may be invisible in formal KCD1 artifacts
High workload variance	More runs are needed for precise small-effect estimates
eBPF permissions	Live tracing usually requires privileges
Policy-gated actions	Some tuning families are suggest-only or blocked by default

11.2 Future Work

Future work can be grouped into four areas.

Table 12. Future-work areas:

Area	Examples
Measurement improvements	IRQ attribution, KMS/DRM fence correlation, wakeup replacement interpretation
Tuning improvements	More granular profiles, safer templates, profile-plan before tune by default
Reporting improvements	Visual reports, confidence summaries, easier artifact navigation
Evaluation improvements	More games, more hardware, more schedulers, controlled variable isolation

Measurement improvements:

- IRQ attribution.
- KMS/DRM fence correlation.
- Better wakeup replacement interpretation.
- More robust foreground detection in Gamescope/Wayland setups.

Tuning improvements:

- More granular profile hypotheses.
- Safer suggested profile templates.
- `profile-plan` before tune by default.
- More adaptive sample-size planning.

Reporting improvements:

- Better visual reports.
- More explicit confidence summaries.
- Easier artifact navigation.
- More direct FYP-style narrative generation from artifacts.

Evaluation improvements:

- More games.
- More hardware.
- More schedulers.
- More controlled variable-isolation experiments.

12. Conclusion

Linux game tuning is noisy, multi-layered, and often anecdotal. This project shows a more conservative approach. `stutter` collects scheduler and frame evidence, generates scoped tuning hypotheses, validates them with repeated measurement, and declines unsupported recommendations.

The KCD1 case study is the central evidence. `stutter` captured a real Proton/Wine/Gamescope workload, produced a plausible CPU-affinity profile, tested it against `baseline-online`, and did not recommend it when the A/B data failed to support it. The experiment validates the workflow rather than the specific CPU-affinity tweak.

The five-run A/B test was sufficient to avoid recommending this unsupported profile, but the generated sample-size estimates show that much larger run counts may be needed to estimate small tuning effects precisely in noisy open-world workloads.

The project therefore succeeds not by finding a magic KCD1 tweak, but by showing that a Linux game-tuning tool can behave conservatively: collect evidence, test a plausible hypothesis, and decline to recommend it when repeated measurements do not support it. That conservative evidence-based behavior is the central contribution of `stutter`.

13. References

External technical references are listed in an IEEE-style numeric format below. Project artifacts are kept separate in Appendix C so that background references and experiment evidence remain distinct.

[1] Linux kernel documentation, “BPF Documentation,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/bpf/

[2] Linux kernel documentation, “BPF maps,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/bpf/maps.html

[3] Linux kernel documentation, “BPF ring buffer,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/bpf/ringbuf.html

[4] Linux kernel documentation, “Scheduler,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/scheduler/index.html

[5] Linux kernel documentation, “CFS Scheduler,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html

[6] Linux kernel documentation, “EEVDF Scheduler,” Linux Kernel Documentation. Accessed: Jun. 5, 2026. [Online]. Available: https://docs.kernel.org/scheduler/sched-eevdf.html

[7] WineHQ, “What is Wine?” Accessed: Jun. 5, 2026. [Online]. Available: https://www.winehq.org/

[8] ValveSoftware, “Proton,” GitHub repository. Accessed: Jun. 5, 2026. [Online]. Available: https://github.com/ValveSoftware/Proton

[9] doitsujin, “DXVK,” GitHub repository. Accessed: Jun. 5, 2026. [Online]. Available: https://github.com/doitsujin/dxvk

[10] ValveSoftware, “gamescope,” GitHub repository. Accessed: Jun. 5, 2026. [Online]. Available: https://github.com/ValveSoftware/gamescope

[11] flightlessmango, “MangoHud,” GitHub repository. Accessed: Jun. 5, 2026. [Online]. Available: https://github.com/flightlessmango/MangoHud

[12] Mesa project, “Mesa 3D Graphics Library documentation.” Accessed: Jun. 5, 2026. [Online]. Available: https://docs.mesa3d.org/

[13] Khronos Group, “Vulkan.” Accessed: Jun. 5, 2026. [Online]. Available: https://www.khronos.org/vulkan/

[14] NIST/SEMATECH, “Engineering Statistics Handbook: Confidence intervals.” Accessed: Jun. 5, 2026. [Online]. Available: https://www.itl.nist.gov/div898/handbook/prc/section1/prc14.htm

[15] NIST/SEMATECH, “Engineering Statistics Handbook: Measurement process characterization.” Accessed: Jun. 5, 2026. [Online]. Available: https://www.itl.nist.gov/div898/handbook/mpc/mpc.htm

[16] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” OOPSLA 2007. Accessed: Jun. 5, 2026. [Online]. Available: https://doi.org/10.1145/1297027.1297033

14. Appendices

Appendix A: Commands and Validation Checks

The KCD1 archive records command shapes rather than every exact shell invocation. Live PIDs were re-detected before recording.

The command examples below preserve the original archive/reproduction layout. In the cleaned Git checkout, `${KCD1_RAW}` refers to the extracted raw KCD1 release archive’s `reports/kcd1-case-study/` directory.

Baseline record shape:

```
stutter record \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --duration 180 \
  --run-name kcd1-rattay-baseline-XX \
  --scenario kcd1-rattay-route-1 \
  --workload-label kcd1-proton-ge-10-34 \
  --route-label rattay-fixed-route-1 \
  --out-dir ${KCD1_RAW}/runs/baseline-XX \
  --mangohud-log <KingdomCome_MANGOHUD_CSV> \
  --hwmon \
  --cpu-freq \
  --runtime-slices \
  --foreground-window \
  --foreground-source sway
```

Tune shape:

```
stutter tune \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profiles ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --runs 5 \
  --baseline-profile baseline-online \
  --warmup-seconds 90 \
  --epoch-seconds 270 \
```

```
  --scenario kcd1-rattay-route-1 \
  --workload-label kcd1-proton-ge-10-34 \
  --route-label rattay-fixed-route-1 \
  --mangohud-log <KingdomCome_MANGOHUD_CSV> \
  --hwmon \
  --out-dir ${KCD1_RAW}/tune/kcd1-affinity-02
```

Profile-plan shape:

```
stutter profile-plan \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profile ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --profile-name kcd1-game-on-1-5-7-11-gamescope-on-0-6
```

Explainable dry-run shape:

```
stutter apply-profile \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profile ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --profile-name kcd1-game-on-1-5-7-11-gamescope-on-0-6 \
  --dry-run \
  --explain
```

Secondary fix-validation recommend shape:

The primary profile-vs-profile conclusion is based on `${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`; the fix-validation artifacts under `${KCD1_RAW}/results/` are secondary and are marked `InvalidExperiment`.

```
stutter recommend \
  --fix-plan ${KCD1_RAW}/fix-plan-cpu-affinity-profile.json \
  --baseline ${KCD1_RAW}/runs/baseline-01 \
  --baseline ${KCD1_RAW}/runs/baseline-02 \
  --baseline ${KCD1_RAW}/runs/baseline-03 \
  --baseline ${KCD1_RAW}/runs/baseline-04 \
  --baseline ${KCD1_RAW}/runs/baseline-05 \
  --tune ${KCD1_RAW}/tune/kcd1-affinity-02 \
  --markdown ${KCD1_RAW}/results/kcd1-fix-validation.md \
  --html ${KCD1_RAW}/results/kcd1-fix-validation.html
```

The JSON version was produced by the same secondary fix-validation command with `--json` redirected to `${KCD1_RAW}/results/kcd1-fix-validation.json`:

```
stutter recommend \
  --fix-plan ${KCD1_RAW}/fix-plan-cpu-affinity-profile.json \
  --baseline ${KCD1_RAW}/runs/baseline-01 \
  --baseline ${KCD1_RAW}/runs/baseline-02 \
  --baseline ${KCD1_RAW}/runs/baseline-03 \
  --baseline ${KCD1_RAW}/runs/baseline-04 \
  --baseline ${KCD1_RAW}/runs/baseline-05 \
  --tune ${KCD1_RAW}/tune/kcd1-affinity-02 \
  --json \
  > ${KCD1_RAW}/results/kcd1-fix-validation.json
```

Validation flow for submission/export:

```
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo fmt --all -- --check
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo test --all
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo clippy --all-targets -- -D warnings
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo run -p xtask -- fixture-check
```

Appendix B: CPU-Affinity Profile TOML

```
[[profile]]
name = "baseline-online"
```

```toml
[[profile.rules]]
affinity = "online"
match_class = ["Game", "GameHelper", "WineServer", "GameScope", "Compositor", "Launcher", "Ste

[[profile]]
name = "kcd1-game-on-1-5-7-11-gamescope-on-0-6"

[[profile.rules]]
affinity = "1-5,7-11"
match_comm = ["Main"]

[[profile.rules]]
affinity = "1-5,7-11"
match_class = ["Game", "GameHelper", "WineServer"]

[[profile.rules]]
affinity = "0,6"
match_class = ["GameScope", "Compositor", "Launcher", "SteamRuntime"]
```

Appendix C: Artifact Map and Evidence Matrix

Table A1. Artifact map:

Artifact	Role
`reports/kcd1-case-study/KCD1_EXPERIMENT_REPORT.md`	Detailed KCD1 case-study report
`reports/kcd1-case-study/CASE_STUDY_SUMMARY.md`	KCD1 archive summary
`reports/kcd1-case-study/ARTIFACT_INDEX.md`	Archive map
`${KCD1_RAW}/setup/system-info.txt`	Machine and session context
`${KCD1_RAW}/runs/baseline-*`	Formal baseline run directories
`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`	Primary A/B profile comparison
`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_recommendation.json`	Recommendation and uncertainty data
`${KCD1_RAW}/results/kcd1-fix-validation.json`	Secondary fix-validation artifact; status is `InvalidExperiment`, not the primary tuned-profile conclusion
`${KCD1_RAW}/profiles/kcd1-affinity-profile-plan-summary.json`	Profile explainability summary
`${KCD1_RAW}/drop-counter-pilot/mapfactor-4-comparison.txt`	Measurement-quality investigation
`${KCD1_RAW}/realworld-stack/realworld-stack-summary.csv`	Exploratory clean vs personal-stack comparison
`${KCD1_RAW}/realworld-stack/ARTIFACT_NOTES.md`	Notes on missing raw MangoHud CSVs for two clean exploratory runs
`docs/TUNING_WORKFLOW.md`	Trusted diagnosis-to-validation loop
`docs/SAFETY.md`	Safety, rollback, and privilege model
`docs/FULL_SYSTEM_WATCHER_ARCHITECTURE.md`	Observer/planner/action-runner architecture
`docs/AUTOTUNE_ARCHITECTURE.md`	Controller contract and keep/revert model

Table A2. Evidence matrix:

Claim	Evidence
Five formal baselines were valid	Baseline analysis and postcheck files in `${KCD1_RAW}/runs/`
Frames were ingested	MangoHud CSVs in `${KCD1_RAW}/mangohud/` and baseline frame-correlation artifacts
Timestamp alignment was monotonic	`reports/kcd1-case-study/CASE_STUDY_SUMMARY.md` and baseline analysis artifacts
Profile matched key KCD threads	`${KCD1_RAW}/profiles/kcd1-affinity-profile-plan-summary.json`
`baseline-online` had lower score in all A/B iterations	`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`
Recommendation was `NeedsRetest`	`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_recommendation.json`
Drop counter was not ringbuf failure	`${KCD1_RAW}/drop-counter-pilot/mapfactor-4-comparison.txt`
Personal stack is exploratory	`${KCD1_RAW}/realworld-stack/README.md`, `${KCD1_RAW}/realworld-stack/setup/launch-options.md`
Build/test flow exists	`${KCD1_RAW}/setup/build-check.txt`, validation commands in Appendix A

Appendix D: Reproducibility Checklist

- Use the same Rattay route and save described in the KCD1 method notes.
- Use Steam with GE-Proton10-34.
- Use the stripped-down measurement configuration: Gamescope and MangoHud logging, no RADV experimental flags, no FSR/FSR4, no gamemode, no mimalloc, and no forced Wine CPU topology.
- Keep `+exec user.cfg` enabled because the archived config is part of the workload.
- Use 1920x1080 through Gamescope, 100 Hz output, and a 100 FPS MangoHud cap.
- Use 180 seconds per measured run.
- Use 270 seconds per tune epoch: 90 seconds of warmup followed by 180 seconds of measurement.
- Re-detect the live Gamescope/KCD process-tree root before each recording.
- Keep background load stable.
- Treat hardware differences as a limitation.

Appendix E: Selected Build/Test Output

The latest validation flow for this report was refreshed on 2026-06-07:

Command	Result
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo metadata --no-deps --format-version 1`	passed
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo fmt --all -- --check`	passed
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 bash scripts/smoke/advisor_offline.sh`	passed
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo run -p xtask -- ci`	passed
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo run -p xtask -- fixture-check`	passed
`RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo run -p xtask -- dependency-hygiene`	passed
`cd evidence-bundle && sha256sum -c MANIFEST.sha256`	passed

Selected output:

- `xtask ci`: formatting, no-allow-attrs, build, clippy, workspace tests, offline recommendation smoke, advisor offline smoke, and architecture tests all passed.
- `advisor_offline.sh`: the JSON smoke path now asserts the current advisor output schema version, `schema_version = 2`.
- `xtask fixture-check`: 20 real fixtures, 19 synthetic fixtures, 20 distinct sanitized capture ids, no missing fixtures, no maturity warnings, and no privacy warnings.
- validation-corpus stage inside `fixture-check`: 44 passed, 0 failed, and 2 ignored.
- `xtask dependency-hygiene`: cargo-deny advisories, bans, licenses, and sources passed; duplicate dependency families matched the approved baseline.
- `evidence-bundle/MANIFEST.sha256`: all curated KCD1 evidence files verified.

Full command logs should stay in repository artifacts or build logs rather than being pasted into the main report.

Appendix F: Additional KCD1 Tables Available in the Artifact Archive

The following supporting tables are available in the repository artifacts and can be used for further inspection or presentation:

- full baseline frame table with P95 and data-quality columns;
- full tune candidate statistics;
- profile-plan per-rule top comms;
- drop-counter pilot command output;
- real-world stack per-run details;
- build/check command output.

Appendix G: Glossary

Term	Meaning
A/B test	Repeated comparison between baseline and tuned conditions
Baseline-online	A tune profile that keeps relevant tasks on the online CPU mask
Diagnostic score	Internal weighted penalty score for comparable runs; lower is better
eBPF	Linux in-kernel instrumentation mechanism used for live tracing
Frame pacing	Consistency of frame delivery over time, especially tail latency
Gamescope	Gaming-focused Wayland compositor often used around Proton games
MangoHud	Overlay and logging tool used here for frame timing
Profile-plan	`stutter` output explaining how profile rules match live tasks
Runnable latency	Delay between a task becoming runnable and receiving CPU time
Tuning hypothesis	A scoped, testable proposal generated from evidence